LOKR

Vault

SMART CONTRACT AUDIT REPORT

IMMUNEBYTES

22 February 2023

Introduction **3**

About Lokr 3

About ImmuneBytes 3

Documentation Details **4**

Audit Goals **4**

Audit Process & Methodology **5**

Audit Details **6**

Security Level References **7**

Finding - V1 8

Finding - V2 9

Audit Changes - V1 **10**

Additional Changes - V1 **13**

Audit Changes - V2 **14**

Additional Changes - V2 **21**

Concluding Remarks **32**

Disclaimer **32**

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Introduction

About Lokr

Lokr provides projects and individuals with a full suite of tools to manage their token economies. Whether it be token creation, distribution, creating cross-chain bridging solutions, or trustless escrows, Lokr has you covered. Lokr lowers the barrier of entry into the crypto space, making blockchain accessible to everyone through simple plug-and-play cross-chain solutions.

Vault:

Vault is a highly-customizable, multi-chain token-locking platform. Designed with the end-user in mind, users can create complex token release models within a few clicks. Vault takes advantage of the latest developments in blockchain technology and, through integration with Oracles, allows users to create event-driven distribution models. The Vault dashboard has a simple and intuitive interface that lets users monitor project token emissions and track their portfolios. Through Vault, projects and users can lock and automate the distribution of tokens, liquidity, and NFTs.

Visit https://www.lokr.io/ to know more about it.

About ImmuneBytes

ImmuneBytes is a security start-up that provides professional services in the blockchain space. The team has hands-on experience conducting smart contract audits, penetration testing, and security consulting. ImmuneBytes's security auditors have worked on various A-league projects and understand DeFi projects like AAVE, Compound, 0x Protocol, Uniswap, and dydx.

The team has secured 205+ blockchain projects by providing security services on different frameworks. The ImmuneBytes team helps start-ups with detailed system analysis, ensuring security and managing the overall project.

Visit http://immunebytes.com/ to learn more about the services.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Documentation Details

The Lokr team has provided the following doc for audit:

1. https://immunebytes.notion.site/Standard-Contract-LOKR

Audit Goals

The focus of the audit was to verify that the smart contract system is secure, resilient, and working according to its specifications. The audit activities can be grouped into the following three categories:

1. Security: Identifying security-related issues within each contract and within the system of contracts.
2. Sound Architecture: Evaluation of the architecture of this system through the lens of established smart contract best practices and general software best practices.
3. Code Correctness and Quality: A full review of the contract source code. The primary areas of focus include

    a. Correctness
    b. Readability
    c. Sections of code with high complexity
    d. Quantity and quality of test coverage

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Audit Process & Methodology

ImmuneBytes team has performed thorough testing of the project starting with analyzing the code design patterns in which we reviewed the smart contract architecture to ensure it is structured and safe use of third-party smart contracts and libraries.

Our team then performed a formal line-by-line inspection of the Smart Contract to find any potential issues like Signature Replay Attacks, Unchecked External Calls, External Contract Referencing, Variable Shadowing, Race conditions, Transaction-ordering dependence, timestamp dependence, DoS attacks, and others.

In the Unit testing phase, we run unit tests written by the developer to verify the functions work as intended. In Automated Testing, we tested the Smart Contract with our in-house developed tools to identify vulnerabilities and security flaws.

The code was audited by a team of independent auditors which includes -

1. Testing the functionality of the Smart Contract to determine proper logic has been followed throughout.
2. Analyzing the complexity of the code by thorough, manual review of the code, line-by-line.
3. Deploying the code on testnet using multiple clients to run live tests.
4. Analyzing failure preparations to check how the Smart Contract performs in case of bugs and vulnerabilities.
5. Checking whether all the libraries used in the code are on the latest version.
6. Analyzing the security of the on-chain data.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Audit Details

Project Name	Lokr
Platform	Ethereum, BSC, Polygon
Languages	Solidity
GitHub Link	https://github.com/polkalokr/lkr-lock-app/tree/pre-deploy
Commit - Final Audit	a4b49bd53088eb1be0dd3cba76cd184adfef0025
Platforms & Tools	Remix IDE, Truffle, VScode, Contract Library, Slither, SmartCheck, Fuzz

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Security Level References

Every issue in this report was assigned a severity level from the following:

CRITICAL

Issues may result in fund leakage or incorrect fund allocation.

HIGH

Issues affecting the business logic, performance, and functionality.

MEDIUM

Issues could lead to data loss or other manipulations.

LOW

Issues around smart contract code upgradability, libraries, and others.

INFORMATIONAL

Issues which can further improve the code on gas optimizations and reusability.

Issues	Critical	High	Medium	Low	Informational
Open	-	-	-	-	-
Closed	1	1	1	7	18
Acknowledged	-	-	-	-	-

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Finding - V1

#	Findings	Risk	Status
1.	Illogical check leading to unreachable data	Critical	Fixed
2.	Incorrect logic leads to design flaw	High	Fixed
3.	Insufficient data sanitization	Medium	Fixed
4.	Emit appropriate events	Low	Fixed
5.	Misleading error message	Low	Fixed
6.	Remove unused functions	Informational	Fixed
7.	Remove commented code	Informational	Fixed
8.	For loop gas optimizations	Informational	Fixed
9.	Code style to increase readability	Informational	Fixed
10.	Spell checks in codebase	Informational	Fixed
11.	Use linting tool for codebase	Informational	Fixed
12.	Use integer LITERAL(s)	Informational	Fixed
13.	Separate functions for code readability	Informational	Fixed
14.	Improve code layout and code style	Informational	Fixed
15.	Correction in NatSpec	Informational	Fixed
16.	Mark immutable variables	Informational	Fixed
17.	Missing of Override keyword from the function	Informational	Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Finding - V2

#	Findings	Risk	Status
1.	LiquidityLock: Violation of Check_Effects_Interaction Pattern in the Withdraw function	Low	Fixed
2.	LockFactory: Absence of Zero Address Validation	Low	Fixed
3.	DeployFee: Multiplication is being performed on the result of Division	Low	Fixed
4.	FixedValueLock: The return value of an External Call is not used effectively	Low	Fixed
5.	LiquidityLock, BaseGovernanceWithUserUpgradable: Lack of adequate input validations before initializing crucial addresses	Low	Fixed
6.	DeployFee: Error messages in require statements can be improved.	Informational	Fixed
7.	FixedValueLock: Redundant comparisons to boolean Constants	Informational	Fixed
8.	DeployFee, LockFactory: Code Style issues	Informational	Fixed
9.	Unlocked Pragma statements found in the contracts	Informational	Fixed
10.	Lock.sol, FixedValueLock.sol - Unused state variables found	Informational	Fixed
11.	Lock.sol: Line- 234, 248 -> Comparison to boolean constant can be avoided.	Informational	Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Audit Changes - V1

The LOKR team has implemented the recommendations based on the auditor's initial finding:

File Changes

1. **Contracts/DateUnlockSchedule.sol (Low Risk issue # 2, Informatory issues # 3,5 and 6):**
    - Add variable at line #12.
    - Update require statement at line # 42.
    - Update for loop at line # 65.
    - Add some conditions at line # 69-72.
    - Replace 1e18 to variable name EXP at line # 74.
    - Update for loop at line # 101.
    - Replace 1e18 to variable name EXP at line # 107.
    - Update for loop at line # 111.

2. **Contracts/Deployfee.sol(Informatory issues #3,5 and 6, Critical issue #1, Medium Risk issue #1);**
    - Update code from line # 70 to 90.
    - Create new function with name updateDeployFeeBeneficiaryInternal()--this function is used to update the address of beneficiary to bring deploy fee at line # 124.
    - Update for loop at line # 266.
    - Add some check from line # 283- 285.
    - Replace for loop with if statement at line # 291.
    - Update for loop at line # 314.
    - Add some check from line # 332- 334.
    - Create new function with name updateCryptoFeedInternal()--This function update the price feed from oracle at line # 338.

3. **Contract/DeplositManger.sol(Informatory issues # 3,5, 6 and 7,12):**
    - Add two variable at line # 28 and 29.
    - Update for loop from line # 87-102.
    - Update for loop from line # 133-139.
    - Update for loop from line # 198-202.
    - Update for loop from line # 221-227.
    - Update for loop from line # 247-253.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

4. **Contract/DeplositMangerMT.sol (Informatory issues # 3, 5, 6, and 7):**
    - Add two variables at line # 30 and 31.
    - Update for loop from line # 93-108.
    - Update for loop from line # 144-150.
    - Update for loop from line # 219-223.
    - Update for loop from line # 242-248.
    - Update for loop from line # 268-274.
    - Update for loop from line # 300-304.
    - Update for loop from line # 310-322.
    - Update for loop from line # 332-342.

5. **Contracts/EventUnlockSchedule.sol (Informatory issues # 3, 5 and 6)**
    - Update for loop from line # 121-125
    - Update for loop from line # 131-135.
    - Update for loop from line # 165 and 181 .
    - Update for loop from line # 259 and 263.

6. **Contracts/FixedValueDepsoirManager.sol(Informatory issues # 3,5,6, and 7):**
    - Add two variables at line # 30 and 31
    - Add some code in addDeposits function at line # 78 and 79.
    - Variable assigning value changed at line # 84.
    - Update for loop from line # 88- 103.
    - Update for loop from line # 134-140.
    - Update _checkAmount function at line # 197.
    - Update for loop at line # 222 and 226.
    - Update for loop from line # 248-254.

7. **Contracts/FixedvalueLock.sol(Low Issue # 1,Informatory issues # 2,3, 5, 6 and 10)**
    - Add two new events at line # 24 and 26.
    - Add code at line # 146.
    - Update for loop at line # 202 and 210
    - Update addBeneficiaries at line # 227-232.
    - Update removeBeneficiaries at line # 243 and 257.
    - Add code at line # 270.
    - Update for loop from line # 314-319

8. **Contracts/IntervalUnlockSchedule.sol (Informatory issues # 3,5 and 6)**
    - Update for loop from line # 65-70.
    - Update for loop from line # 99-103.

- Update for loop from line # 109-115.

9. **Contracts/Lock.sol(Low risk issue # 1, Informatory issues # 1, 2, 3, 5, 6 and 10)**
    - Add two new events at line # 22 and 24.
    - Update for loop from line # 197 and 205.
    - Update addBeneficiaries at line # 222-227.
    - Update removeBeneficiaries at line # 238 and 252.
    - Add code at line # 265.
    - Update for loop from line # 309-314.

10. **Contracts/LockFactory.sol (Informatory issues # 3, 5, 6 and 8):**
    - Update for loop from line # 54-59.
    - Add new function getCryptoPriceFeed at line # 111.
    - Update for loop from line # 162-168.
    - Update for loop from line # 176-182.
    - Add new code at line # 248.
    - Add new code from line # 393-444.

11. **Contracts/SplitManagger.sol (Informatory issues # 3, 5,6 and 11)**
    - Update require statement at line # 41.
    - Update for loop from line # 86-90.
    - Update for loop from line # 110-114.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Additional Changes - V1

LOKR Changes With TX-fees

The changes we made in this section is if we mint a token on Mintr platform with Tx fee and any user (token owner or holder) wants to create a lock with a Lokr then the Tx fee will be whitelisted (means no Tx fee will be deducted in scenario of either creating a lock or beneficiary claiming the lock). However if any token is minted from another platform with Tx fee capability then user cannot create a lock on this platform.

File changes

- **Contracts/FixedValueLock.sol :**
  Add require Statement on line # 147 in this function **_initializeBeneficiaries()**.

Note: This requires statements to check that those token which have tx-fee capabilities can't lock at vault.

- **Contracts/Lock.sol:**
  Require statement added at on line # 146:
  require(tokenERC20.balanceOf(address(*this*)) == _lockedAmount, "ERROR: Can't Lock Tokens With TX Fee");

Note: This requires statements to check that those token which have tx-fee capabilities can't lock at vault.

- **Contracts/LockFactory.sol:**
  - Add Tx-fee-whitelisted role at line # 36.
  - Add address type variable at line # 40.
  - Move code from line # 32-45 at line # 16-29.
  - Create new function with name setMintrFactory() at line # 102.
  - Add some changes and checks from line # 371 to 379
  - Create new function with name whitelistLokr() at line # 461.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Audit Changes - V2

The LOKR team has implemented the recommendations based on the auditor's initial finding:

Low risk issues & Informatory issues

- Solution on line 261 & 265 adding the safeERC20

```
258     -   TransferHelper.safeTransferFrom(_lpToken, address(msg.sender), address(this), _amount);
    261 +   SafeERC20.safeTransferFrom(IERC20(_lpToken), address(msg.sender), address(this), _amount);
259 262
260 263     // send univ2 fee to dev address
261 264     uint256 liquidityFee = _amount.mul(gFees.liquidityFee).div(1000);
262     -   TransferHelper.safeTransfer(_lpToken, devaddr, liquidityFee);
    265 +   SafeERC20.safeTransfer(IERC20(_lpToken), devaddr, liquidityFee);
```

- Solution on line 107

```
    107 +       require(_mintrContract != address(0), "Can't set address 0");
```

- Acknowledged
- Solution on lines 304-305

```
304     -       splitManager.registerSplit(originId, newIDs, splitParts);
    304 +       bool success = splitManager.registerSplit(originId, newIDs, splitParts);
    305 +       require(success, "ERROR: You can not Split this NFT");
```

- Solution on line 80 LiquidityLock

```
79  79      function setDev(address payable _devaddr) public onlyOwner {
    80  +     require(_devaddr != address(0), "Can not set Address 0");
80  81        devaddr = _devaddr;
81  82      }
```

- Solution on line 50 BaseGovernanceWithUserUpgradeable

```
49  49  +   function proposeNewImplementation(address implementationAddress) external onlyRole(GOVERNANCE_ROLE) {
    50  +       require(implementationAddress != address(0), "Can not set Address(0)");
50  51          _proposedUpgrade = implementationAddress;
51  52      }
```

- Solution on line 97

```
97      -   "ERROR: Deploy Fee Cant be 0"
    97  +   "Percentage fee and Fixed fee cant be 0"
```

- Solution on lines 60 and 229

```
60      -   require(firstReleaseReached == false, "First release was reached, can't add/remove new beneficiaries");
    60  +   require(!firstReleaseReached, "First release reached, cant add/remove new beneficiaries");
```

```
229     -   require(canAddBeneficiaries == true, "ERROR: Cant add new beneficiaries");
    229 +   require(canAddBeneficiaries, "ERROR: Cant add new beneficiaries");
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

- Solutions on all the contracts to block the version to 0.8.17

```
2 contracts/DateUnlockSchedule.sol
@@ -1,6 +1,6 @@
// contracts/UnlockManager.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.2;
+ pragma solidity 0.8.17;
```

```
6 contracts/DeployFee.sol
@@ -1,5 +1,5 @@
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

```
2 contracts/DepositManager.sol
@@ -1,6 +1,6 @@
// contracts/DepositManager.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

```
2 contracts/DepositManagerMT.sol
@@ -1,6 +1,6 @@
// contracts/DepositManager.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

```
2 contracts/EqualUnlockSchedule.sol
@@ -1,6 +1,6 @@
// contracts/UnlockManager.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.2;
+ pragma solidity 0.8.17;
```

```
2 contracts/EventUnlockSchedule.sol
@@ -1,5 +1,5 @@
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.7;
+ pragma solidity 0.8.17;
```

```
2 contracts/FixedValueDepositManager.sol
@@ -1,6 +1,6 @@
// contracts/DepositManager.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

9 contracts/FixedValueLock.sol

```
@@ -1,6 +1,6 @@
// contracts/Lock.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

4 contracts/Lock.sol

```
@@ -1,6 +1,6 @@
// contracts/Lock.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

3 contracts/LockFactory.sol

```
@@ -1,5 +1,5 @@
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

2 contracts/SplitManager.sol

```
@@ -1,6 +1,6 @@
// contracts/SplitManager.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;

import "./common/BaseGovernanceWithUserUpgradable.sol";
import "./interfaces/ILock.sol";
```

2 contracts/SplitManagerFalse.sol

```
@@ -1,6 +1,6 @@
// contracts/SplitManager.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;

import "./common/BaseGovernanceWithUserUpgradable.sol";
import "./interfaces/ILock.sol";
```

```diff
contracts/SplitManagerTrue.sol
@@ -1,6 +1,6 @@
// contracts/SplitManager.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;

import "./common/BaseGovernanceWithUserUpgradable.sol";
import "./interfaces/ILock.sol";
```

```diff
contracts/UpkeepConsumer.sol
@@ -1,5 +1,5 @@
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.6;
+ pragma solidity 0.8.17;
```

```diff
contracts/access/AccessControlRewrited.sol
@@ -1,6 +1,6 @@
// SPDX-License-Identifier: BUSL-1.1

- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;

import "@openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/utils/StringsUpgradeable.sol";
```

```diff
contracts/common/BaseGovernanceUpgradable.sol
@@ -1,5 +1,5 @@
// SPDX-License-Identifier: Unlicensed
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

```diff
contracts/common/BaseGovernanceWithUserUpgradable.sol
@@ -1,5 +1,5 @@
// SPDX-License-Identifier: Unlicensed
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

2 contracts/interfaces/IDepositManager.sol

```diff
@@ -1,6 +1,6 @@
// contracts/interfaces/IDepositManager.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;

interface IDepositManager {
    function LOCK_CONTRACT_ROLE() external view returns (bytes32);
```

9 contracts/interfaces/IERC20Burnable.sol

```diff
@@ -1,10 +1,9 @@
// SPDX-License-Identifier: MIT

- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

2 contracts/interfaces/ILock.sol

```diff
@@ -1,6 +1,6 @@
// contracts/interfaces/ILock.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

2 contracts/interfaces/ISplitManager.sol

```diff
@@ -1,6 +1,6 @@
// contracts/interfaces/ISplitManager.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;

interface ISplitManager {
    /**
```

2 contracts/interfaces/IUnlockSchedule.sol

```diff
@@ -1,6 +1,6 @@
// contracts/interfaces/IUnlockSchedule.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

2 contracts/security/PausableRewrited.sol

```
@@ -1,6 +1,6 @@
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
import "@openzeppelin/contracts/utils/Context.sol";
import "@openzeppelin/contracts/utils/StorageSlot.sol";
```

2 contracts/security/PausableUpgradeableRewrited.sol

```
@@ -1,6 +1,6 @@
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

2 contracts/test/FixedValueLockMock.sol

```
@@ -1,6 +1,6 @@
// contracts/Lock.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
import "@openzeppelin/contracts-upgradeable/token/ERC721/ERC721Upgradeable.sol";
import "@openzeppelin/contracts-upgradeable/token/ERC20/IERC20Upgradeable.sol";
```

2 contracts/test/LockMock.sol

```
@@ -1,6 +1,6 @@
// contracts/Lock.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

2 contracts/test/PolkalokrTestToken.sol

```
@@ -1,6 +1,6 @@
// contracts/PolkalokrTestToken.sol
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;

import "@openzeppelin/contracts-upgradeable/token/ERC20/ERC20Upgradeable.sol";
import "@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";
```

2 contracts/test/TestERC20.sol

```
@@ -1,5 +1,5 @@
// SPDX-License-Identifier: BUSL-1.1
- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

2 contracts/utils/ArrayStorageSlot.sol

```
@@ -1,6 +1,6 @@
// SPDX-License-Identifier: BUSL-1.1

- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;

library ArrayStorageSlot {
    struct AddressSlot {
```

2 contracts/utils/MappingStorageSlot.sol

```
@@ -1,6 +1,6 @@
// SPDX-License-Identifier: BUSL-1.1

- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

2 contracts/utils/TransferHelper.sol

```
@@ -1,6 +1,6 @@
// SPDX-License-Identifier: MIT

- pragma solidity ^0.8.0;
+ pragma solidity 0.8.17;
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Additional Changes - V2

Modified Functions:

1. mintNFTs()
    a. Added **Minted** event for newly minted IDs and owner addresses
    b. Added **uint256[] Return** type to get minted IDs

2. split()
    a. Added **uint256[] Return** type to get newly minted IDs after splitting

Adding the Discount token

Modified Contracts:

1. LockFactory
2. DeployFee

New Variables:

1. discountToken - This is the address of the token
2. discountAmount - This is the amount to be discounted 1e18 is 100% of discount
3. discountTokenPriceFeed - The priceFeed of the token

New Functions:

1. setDiscountTokenInternal - Internal Function on DeployFee to set the discount token
2. setDiscountToken - External function on LockFactory to set the discount token

Modified Functions:

1. calculateRequiredTokens
    a. If the payment token is the discount token, will calculate the discount and will send that amount to the lockfactory

Discount Token

New Mappings on DeployFee:

```
+    mapping(address => uint256) internal discountAmount;
```

```
+    mapping(address => bool) internal isDiscountToken;
```

Add a conditional on CalculateRequiredTokens

```
+        if(isDiscountToken[paymentTokenAddress]) {
+            fixedTokenAmount -= ( fixedTokenAmount * discountAmount[paymentTokenAddress] / EXP_VALUE );
+            percentageTokenAmount -= ( percentageTokenAmount * discountAmount[paymentTokenAddress] / EXP_VALUE );
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

New Functions setDiscountToken and removeDiscountToken

```
267  269      function setDiscountTokenInternal(
268  270          address _discountToken,
269       -      address _discountPriceFeed,
270  271          uint256 _discountAmount
271  272      ) internal {
```

```
273  +          require(_discountToken != address(0), "Cant set address 0");
274  +          require(_discountAmount < EXP_VALUE, "Discount cant be more than 100%");
275  +          require(isPaymentToken[_discountToken], "This token is not supported");
276  +          isDiscountToken[_discountToken] = true;
277  +          discountAmount[_discountToken] = _discountAmount;
278  +      }
279  +
280  +      function removeDiscountTokenInternal(
281  +          address _discountToken
282  +      ) internal {
283  +          require(isDiscountToken[_discountToken], "This is not a discount token");
284  +          delete isDiscountToken[_discountToken];
285  +          delete discountAmount[_discountToken];
```

\

SetDiscountToken in LockFactory

```
     169  +      function setDiscountTokens(
     170  +          address[] memory _discountTokens,
     171  +          uint256[] memory _discountAmounts
115  172      ) external onlyRole(FACTORY_MANAGER) {
116       -          setDiscountTokenInternal(_discountToken, _discountPriceFeed, _discountAmount);
     173  +          require(_discountAmounts.length == _discountTokens.length, "Tokens and Amounts dont have same length");
     174  +          for(uint256 i; i < _discountAmounts.length; ) {
     175  +              setDiscountTokenInternal(_discountTokens[i], _discountAmounts[i]);
     176  +              unchecked {
     177  +                  i++;
     178  +              }
     179  +          }
```

Events emitted when contracts are initialised

DepositManager

```
9 contracts/DepositManager.sol → contracts/DepositManager/DepositManager.sol                Viewed
@@ -30,6 +30,8 @@ contract DepositManager is BaseGovernanceWithUserUpgradable, IDepositManager {
    uint256 internal constant SQREXP = 1e36;

    bytes32 public constant LOCK_CONTRACT_ROLE = keccak256("LOCK_CONTRACT_ROLE");
+
+   event DepositManagerInitialized(address);

    modifier onlyLockContract() {
        require(hasRole(LOCK_CONTRACT_ROLE, _msgSender()), "ERROR: Only lock");
@@ -50,6 +52,7 @@ contract DepositManager is BaseGovernanceWithUserUpgradable, IDepositManager {
        lockContract = _lock;
        // schedule = IUnlockSchedule(_lock.schedule()); // THis will fail because we initialize DM before the Lock
        _setupRole(LOCK_CONTRACT_ROLE, address(_lock));
+       emit DepositManagerInitialized(address(this));
    }
```

DepositManagerMT

```
9 contracts/DepositManagerMT.sol → ...racts/DepositManager/DepositManagerMT.sol                Viewed
 /**
  * @title Deposit manager
@@ -36,6 +36,8 @@ contract DepositManagerMT is BaseGovernanceWithUserUpgradable, IDepositManager {

    mapping (bytes32 => bool) private _isMinted;

+   event DepositManagerMTInitialized(address);
+
    modifier onlyLockContract() {
        require(hasRole(LOCK_CONTRACT_ROLE, _msgSender()), "ERROR: Only lock");
        _;
@@ -55,6 +57,7 @@ contract DepositManagerMT is BaseGovernanceWithUserUpgradable, IDepositManager {
        lockContract = _lock;
        // schedule = IUnlockSchedule(_lock.schedule()); // THis will fail because we initialize DM before the Lock
        _setupRole(LOCK_CONTRACT_ROLE, address(_lock));
+       emit DepositManagerMTInitialized(address(this));
    }
```

FixedValueDepositManager

```
9 contracts/FixedValueDepositManager.sol → ...positManager/FixedValueDepositManager.sol   Viewed
    uint256 internal constant MAXIMUM_LOCK_AMOUNT = 1e41;
    uint256 internal constant SQREXP = 1e36;
+
+   event FixedValueDepositManagerInitialized(address);

    modifier onlyLockContract() {
        require(hasRole(LOCK_CONTRACT_ROLE, _msgSender()), "ERROR: Only lock");
@@ -50,6 +52,7 @@ contract FixedValueDepositManager is BaseGovernanceWithUserUpgradable, IDepositM
        lockContract = _lock;
        // schedule = IUnlockSchedule(_lock.schedule()); // THis will fail because we initialize DM before the Lock
        _setupRole(LOCK_CONTRACT_ROLE, address(_lock));
+       emit FixedValueDepositManagerInitialized(address(this));
```

FixedValueLock

```
31 contracts/FixedValueLock.sol → contracts/Lock/FixedValueLock.sol   Viewed
+   event FixedValueLockInitialized(address);
+
    modifier onlyBeneficiaryManager() {
        require(hasRole(BENEFICIARY_MANAGER_ROLE, _msgSender()), "ERROR: You are not the Beneficiary Manager");
        _;
@@ -141,6 +142,7 @@ contract FixedValueLock is BaseGovernanceWithUserUpgradable, ERC721EnumerableUpg
        _setupRole(BENEFICIARY_MANAGER_ROLE, _msgSender());
        _setupRole(DEPOSIT_MANAGER_ROLE, address(_depositManager));
        _initializeBeneficiaries(beneficiariesData, _lockedAmount);
+       emit FixedValueLockInitialized(address(this));
```

LiquidityLock

```
8 contracts/LiquidityLock.sol → contracts/Lock/LiquidityLock.sol   Viewed
    event onWithdraw(address lpToken, uint256 amount);
+   event LiquidityLockInitialized(address);

    modifier onlyOwner {
      require(hasRole(DEFAULT_ADMIN_ROLE, _msgSender()), "ERROR: Only the Owner");
@@ -72,8 +73,7 @@ contract LiquidityLock is BaseGovernanceWithUserUpgradable, ReentrancyGuardUpgra
      gFees.referralHold = 10e18;
      gFees.referralDiscount = 100; // 10%
      uniswapFactory = _uniswapFactory;
-
-
+     emit LiquidityLockInitialized(address(this));
```

Lock

```
18 contracts/Lock.sol → contracts/Lock/Lock.sol                                Viewed
25  25      event Minted(address[] addresses, uint256[] IDs);
    26  +   event LockInitialized(address);
26  27
27  28      uint256 private totalIDs;
28  29
@@ -135,6 +136,7 @@ contract Lock is BaseGovernanceWithUserUpgradable, ERC721EnumerableUpgradeable {
135 136         _setupRole(BENEFICIARY_MANAGER_ROLE, _msgSender());
136 137         _setupRole(DEPOSIT_MANAGER_ROLE, address(_depositManager));
137 138         _initializeBeneficiaries(beneficiariesData, _lockedAmount);
    139 +       emit LockInitialized(address(this));
```

DateUnlockSchedule

```
20 contracts/DateUnlockSchedule.sol → ...ts/ScheduleManager/DateUnlockSchedule.sol                Viewed
    18  +   event DateUnlockScheduleInitialized(address);
    19  +
18  20      function initialize(ILock lock, bytes calldata data) public initializer {
19  21          __DateUnlockSchedule_init(lock, data);
20  22      }
@@ -47,6 +49,7 @@ contract DateUnlockSchedule is BaseGovernanceWithUserUpgradable, IUnlockSchedule
47  49          normalizedAmounts = _normalizedAmounts;
48  50          unlockTimestamps = _unlockTimestamps;
49  51          ownerCanWithdraw = true;
    52  +       emit DateUnlockScheduleInitialized(address(this));
```

EqualUnlockSchedule

```
22 contracts/EqualUnlockSchedule.sol → ...s/ScheduleManager/EqualUnlockSchedule.sol                Viewed
    19  +   event EqualUnlockScheduleInitialized(address);
    20  +
19  21      function initialize(ILock lock, bytes calldata data) public initializer {
20  22          __EqualUnlockSchedule_init(lock, data);
21  23      }
@@ -52,6 +54,7 @@ contract EqualUnlockSchedule is BaseGovernanceWithUserUpgradable, IUnlockSchedul
52  54          if(lockLenght % unlockPeriod != 0) totalPeriods++; //add possible partial period at the end of lock time
53  55
54  56          ownerCanWithdraw = true;
    57  +       emit EqualUnlockScheduleInitialized(address(this));
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

IntervalUnlockSchedule

```
19 contracts/IntervalUnlockSchedule.sol → ...cheduleManager/IntervalUnlockSchedule.sol   Viewed
+    event IntervalUnlockScheduleInitialized(address);

     function initialize(ILock lock, bytes calldata data) public initializer {
         (
@@ -45,6 +46,7 @@ contract IntervalUnlockSchedule is BaseGovernanceWithUserUpgradable, IUnlockSche
         keyPoints = _keyPoints;
         amounts = _amounts;
         ownerCanWithdraw = true;
+        emit IntervalUnlockScheduleInitialized(address(this));
```

LinearUnlockSchedule

```
17 contracts/LinearUnlockSchedule.sol → .../ScheduleManager/LinearUnlockSchedule.sol   Viewed
+    event LinearUnlockScheduleInitialized(address);
+
     function initialize(ILock lock, bytes calldata data) public initializer {
         (
             uint256 _lockStartTimestamp,
@@ -47,6 +49,7 @@ contract LinearUnlockSchedule is BaseGovernanceWithUserUpgradable, IUnlockSchedu
         lockEnd = _lockEndTimestamp;
         cliffTimestamp = _cliffTimestamp;
         ownerCanWithdraw = true;
+        emit LinearUnlockScheduleInitialized(address(this));
```

SplitManager

```
38 contracts/SplitManager.sol → contracts/SplitManager/SplitManager.sol   Viewed
+    event SplitManagerInitialized(address);

     modifier onlyLockContract() {
         require(hasRole(LOCK_CONTRACT_ROLE, _msgSender()), "ERROR: Only lock");
@@ -43,6 +45,7 @@ contract SplitManager is BaseGovernanceWithUserUpgradable, ISplitManager {
         _setupRole(LOCK_CONTRACT_ROLE, address(_lock));
         lockContract = _lock;
         initialLockedPart = _initialLockedPart;
+        emit SplitManagerInitialized(address(this));
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

SplitManagerFalse

```
15 contracts/SplitManagerFalse.sol → contracts/SplitManager/SplitManagerFalse.sol
+    event SplitManagerFalseInitialized(address);

     modifier onlyLockContract() {
         require(hasRole(LOCK_CONTRACT_ROLE, _msgSender()), "ERROR: Only lock");
@@ -33,6 +35,7 @@ contract SplitManagerFalse is BaseGovernanceWithUserUpgradable, ISplitManager {
     function __SplitManagerFalse_init_unchained(ILock _lock) internal onlyInitializing {
         _setupRole(LOCK_CONTRACT_ROLE, address(_lock));
         lockContract = _lock;
+        emit SplitManagerFalseInitialized(address(this));
```

SplitManagerTrue

```
15 contracts/SplitManagerTrue.sol → contracts/SplitManager/SplitManagerTrue.sol
+
+    event SplitManagerTrueInitialized(address);

     modifier onlyLockContract() {
         require(hasRole(LOCK_CONTRACT_ROLE, _msgSender()), "ERROR: Only lock");
@@ -33,6 +35,7 @@ contract SplitManagerTrue is BaseGovernanceWithUserUpgradable, ISplitManager {
     function __SplitManagerTrue_init_unchained(ILock _lock) internal onlyInitializing {
         _setupRole(LOCK_CONTRACT_ROLE, address(_lock));
         lockContract = _lock;
+        emit SplitManagerTrueInitialized(address(this));
```

New Function on SplitManagers Contracts: getUnlockedPart

ISplitManager

```
13 contracts/interfaces/ISplitManager.sol
+    /**
+     * @notice Get the unlocked part of a NFT/Deposit
+     * @param ID ID-NFT to see the locked part
+     * @return unlockedPart unlocked part proportion normalized
+     */
+    function getUnlockedPart(uint256 ID) view external returns(uint256 unlockedPart);
```

SplitManager

```
38 contracts/SplitManager.sol → contracts/SplitManager/SplitManager.sol
+    /**
+     * @notice Get the locked part of a NFT/Deposit
+     * @param id NFT ID to check the locked part
+     * @return unlockedPart unlocked part proportion normalized
+     */
+    function getUnlockedPart(uint256 id) public view override returns(uint256 unlockedPart) {
+        unlockedPart = TOTAL - getLockedPart(id);
+    }
+
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

SplitManagerFalse

15 contracts/SplitManagerFalse.sol → contracts/SplitManager/SplitManagerFalse.sol Viewed

```
+    /**
+     * @notice Get the locked part of a NFT/Deposit
+     * @return unlockedPart unlocked part proportion normalized
+     */
+    function getUnlockedPart(uint256 /*id*/) public pure override returns(uint256 unlockedPart) {
+        unlockedPart = 0;
+    }
```

SplitManagerTrue

15 contracts/SplitManagerTrue.sol → contracts/SplitManager/SplitManagerTrue.sol Viewed

```
+    /**
+     * @notice Get the locked part of a NFT/Deposit
+     * @return unlockedPart unlocked part proportion normalized
+     */
+    function getUnlockedPart(uint256 /*id*/) public pure override returns(uint256 unlockedPart) {
+        unlockedPart = 1e18;
+    }
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

New Function on ScheduleManagers contracts: nextRelease

IUnlockSchedule

14 contracts/interfaces/IUnlockSchedule.sol

```
 /**
  * @notice Function that get the timestamp when the lock will finish
- * @return Timestamp when the lock end
+ * @return timestamp when the lock end
  */
- function lockEnd() external view returns(uint);
+ function lockEnd() external view returns(uint256 timestamp);
+
+ /**
+  * @notice Function that get the next timestamp when the lock will be claimable
+  * @return timestamp to next release
+  */
+ function nextRelease() external view returns(uint256 timestamp);
```

DateUnlockSchedule

20 contracts/DateUnlockSchedule.sol → ...ts/ScheduleManager/DateUnlockSchedule.sol

```
+ /**
+  * @notice Function that get the next timestamp when the lock will be claimable
+  * @return timestamp to next release
+  */
+ function nextRelease() external view returns(uint256 timestamp) {
+     for(uint256 i; i < unlockTimestamps.length; ) {
+         if(unlockTimestamps[i] > block.timestamp) return timestamp = unlockTimestamps[i];
+         unchecked {
+             i++;
+         }
+     }
+ }
+
```

EqualUnlockSchedule

22 contracts/EqualUnlockSchedule.sol → ...s/ScheduleManager/EqualUnlockSchedule.sol

```
+ /**
+  * @notice Function that get the next timestamp when the lock will be claimable
+  * @return timestamp to next release
+  */
+ function nextRelease() external view returns(uint256 timestamp) {
+     timestamp = lockStart;
+     for(uint256 i; i < totalPeriods; ) {
+         timestamp += unlockPeriod;
+         if(timestamp > block.timestamp) return timestamp;
+         unchecked {
+             i++;
+         }
+     }
+ }
+
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

IntervalUnlockSchedule

19 contracts/IntervalUnlockSchedule.sol → ...cheduleManager/IntervalUnlockSchedule.sol Viewed

```
+    /**
+     * @notice Function that get the next timestamp when the lock will be claimable
+     * @return timestamp to next release
+     */
+    function nextRelease() external view returns(uint256 timestamp) {
+        for(uint256 i; i < keyPoints.length; ) {
+            if(keyPoints[i] > block.timestamp) return timestamp = keyPoints[i];
+            unchecked {
+                i++;
+            }
+        }
+    }
+
```

LinearUnlockSchedule

17 contracts/LinearUnlockSchedule.sol → .../ScheduleManager/LinearUnlockSchedule.sol Viewed

```
+     * @notice Function that get the next timestamp when the lock will be claimable
+     * @return timestamp to next release
+     */
+    function nextRelease() external view returns(uint256 timestamp) {
+        if(block.timestamp > cliffTimestamp) {
+            timestamp = block.timestamp;
+        } else timestamp = cliffTimestamp;
+    }
+
```

New Functions on Lock: getNextReleaseTimestamp and getSpittablePart

Lock

```
18 contracts/Lock.sol → contracts/Lock/Lock.sol                                   Viewed
212  214
     215 +    function getNextReleaseTimestamp() view external returns(uint256) {
     216 +        return schedule.nextRelease();
     217 +    }
     218 +
213  219      function getUnsplittablePart(uint256 id) view external returns(uint256) {
214  220          return splitManager.getLockedPart(id);
215  221      }
216  222
     223 +    function getSplittablePart(uint256 id) view external returns(uint256) {
     224 +        return splitManager.getLockedPart(id);
     225 +    }
     226 +
```

FixedValueLock

```
31 contracts/FixedValueLock.sol → contracts/Lock/FixedValueLock.sol                Viewed
217  220
     221 +    function getNextReleaseTimestamp() view external returns(uint256) {
     222 +        return schedule.nextRelease();
     223 +    }
     224 +
218  225      function getUnsplittablePart(uint256 id) view external returns(uint256) {
219  226          return splitManager.getLockedPart(id);
220  227      }
221  228
     229 +    function getSplittablePart(uint256 id) view external returns(uint256) {
     230 +        return splitManager.getLockedPart(id);
     231 +    }
     232 +
```

New Function on LockFactory: updateDeployFeeBeneficiary

LockFactory

```
     205 +    function updateDeployFeeBeneficiary(address newDeployFeeBeneficiary) external onlyRole(FACTORY_MANAGER) {
     206 +        updateDeployFeeBeneficiaryInternal(newDeployFeeBeneficiary);
140  207      }
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Concluding Remarks

While conducting the audits of the Lokr - Vault smart contracts, it was observed that the contracts contain Critical, High, Medium, and Low severity issues along with a few recommendations.

Our auditors suggest that Critical, High, Medium and Low severity issues should be resolved by the developers. The recommendations given will improve the operations of the smart contract.

Disclaimer

ImmuneBytes's audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore running a bug bounty program as a complement to this audit is strongly recommended.

Our team does not endorse the Lokr platform or its product nor this audit is investment advice.
Notes:

- Please make sure contracts deployed on the mainnet are the ones audited.
- Check for the code refactor by the team on critical issues.

IMMUNEBYTES
AUDITS